

基于卷积神经网络的 MNIST 手写数字识别实验报告

姓名：刘泽睿 学号：PB24000157

2026 年 6 月 16 日

摘要

本实验基于卷积神经网络对 MNIST 手写数字数据集进行分类任务。实验对比了两种不同深度的卷积网络架构（双卷积层的基础结构与引入批量归一化的五卷积层深层结构），并在学习率、权重衰减、丢弃率三个超参数构成的 $2 \times 2 \times 2$ 网格上进行了系统性搜索，共计 16 组对照实验。实验结果表明，深层网络在最佳超参数配置下取得了 **99.57%** 的测试准确率，显著优于基础网络的最佳结果 **99.37%**。本文完整记录了实验设计、模型架构、超参数配置、实验结果与分析，以及调试过程。完整代码已开源至 <https://github.com/mathmonkeyliu/USTC-AI-homework>。

目录

1	引言	2
2	实验设计	2
2.1	数据集	2
2.2	模型架构	3
2.2.1	基础 CNN：双卷积层结构	3
2.2.2	深层 CNN：五卷积层 + 批量归一化结构	3
2.3	架构差异分析	4
2.4	超参数搜索空间	4
2.5	训练配置	4
3	实验结果与分析	5
3.1	总体结果	5
3.2	结果分析	5
3.2.1	模型架构对性能的影响	5
3.2.2	超参数对性能的影响	6
3.3	训练过程准确率曲线	6

1 引言	2
4 调试与错误排查	8
5 自定义手写数字识别实验	9
5.1 模型选择与预测方法	9
5.2 识别结果	10
6 结论	11
A 附录：完整代码	11
A.1 model.py ——模型定义	12
A.2 dataloader.py ——数据加载	14
A.3 train.py ——训练与评估	16
A.4 main.py ——主程序入口	18
A.5 log.py ——日志模块	21
A.6 predict_handwritten.py ——手写数字预测	21
A.7 run_experiments.sh ——批量实验脚本	24

1 引言

手写数字识别是计算机视觉领域的经典入门任务,是模式分类中的基础问题。MNIST 数据集包含 60,000 张训练图像和 10,000 张测试图像,每张为 28×28 的灰度手写数字图像,共 10 个类别 (0-9)。卷积神经网络因其在图像特征提取上的优异表现,已成为 MNIST 任务上的标准方法。

本实验旨在探究不同网络深度及超参数设置对手写数字识别性能的影响。实验对比了两种模型结构:一种为仅含两个卷积层的基础网络,另一种为引入批量归一化的五卷积层深层网络。此外,实验在三个关键超参数上进行了网格搜索——学习率、权重衰减系数和丢弃率,以探索各超参数对模型性能的影响规律。

2 实验设计

2.1 数据集

实验使用 MNIST 手写数字数据集,包含 60,000 张训练图像和 10,000 张测试图像。每张图像为 28×28 像素的单通道灰度图。数据预处理流程如下:

1. 将 PIL 图像转换为 PyTorch Tensor (`transforms.ToTensor()`);
2. 使用 MNIST 数据集的全局均值和标准差进行归一化: `Normalize((0.1307,), (0.3081,))`。

训练时以每批 64 张图像加载数据并随机打乱,测试时不打乱。

2.2 模型架构

实验实现了两种卷积神经网络，二者的核心差异在于网络深度和是否使用批量归一化。

2.2.1 基础 CNN：双卷积层结构

基础网络由一个简单的特征提取器和一个分类器组成，结构如下：

- **特征提取器**：两层卷积，每层后接 ReLU 激活和 2×2 最大池化：

```
Conv2d(1, 32, 3×3, padding=1) → ReLU → MaxPool2d(2) # 28→14  
Conv2d(32, 64, 3×3, padding=1) → ReLU → MaxPool2d(2) # 14→7
```

- **分类器**：全连接层：

```
Flatten → Linear(64×7×7, 128) → ReLU → Dropout(p) → Linear(128, 10)
```

该模型参数量约为 0.42M。

2.2.2 深层 CNN：五卷积层 + 批量归一化结构

深层网络在基础网络之上进行了三方面增强：增加卷积层数、引入批量归一化层、扩展通道数与全连接层宽度。该模型由三个卷积块组成：

- **卷积块一**（双层卷积 + 批量归一化）：

```
Conv2d(1, 32, 3×3) → BN → ReLU  
Conv2d(32, 32, 3×3) → BN → ReLU → MaxPool2d(2) # 28→14
```

- **卷积块二**（双层卷积 + 批量归一化）：

```
Conv2d(32, 64, 3×3) → BN → ReLU  
Conv2d(64, 64, 3×3) → BN → ReLU → MaxPool2d(2) # 14→7
```

- **卷积块三**（单层卷积 + 批量归一化）：

```
Conv2d(64, 128, 3×3) → BN → ReLU → MaxPool2d(2) # 7→3
```

- 分类器：

Flatten $\rightarrow$ Linear(128 $\times$ 3 $\times$ 3, 256) $\rightarrow$ ReLU $\rightarrow$ Dropout(p) $\rightarrow$ Linear(256, 10)

该模型参数量约为 0.55M。

2.3 架构差异分析

相比于基础网络，深层网络从以下几个方面进行了增强：

1. **增加网络深度**：基础网络仅有 2 个卷积层，而深层网络提升至 5 个卷积层（分布在三个卷积块中），显著增强了模型的特征提取能力；
2. **引入批量归一化**：在每个卷积层后添加批量归一化层，有效缓解了内部协变量偏移问题，加速收敛并提升训练稳定性；
3. **扩展卷积通道数**：基础网络的通道数从 32 递增至 64，而深层网络扩展至 32 $\rightarrow$ 64 $\rightarrow$ 128，更宽的通道数使模型能捕捉更丰富的特征表示；
4. **扩大全连接隐藏层**：分类器的隐藏层维度从 128 提升至 256，增强了分类头的非线性表达能力。

这些修改使深层网络具备了更强的表征能力。得益于批量归一化层的引入，深层网络在训练过程中收敛更快、更稳定，且最终取得了明显更高的测试准确率。

2.4 超参数搜索空间

实验对三个关键超参数进行网格搜索，每个超参数设两个取值，构成 $2 \times 2 \times 2 = 8$ 种组合：

- 学习率：{0.0005, 0.001}
- 权重衰减系数：{0.0, 0.0001}
- 丢弃率：{0.3, 0.5}

2.5 训练配置

所有实验共享以下固定配置：

- 优化器：Adam
- 损失函数：交叉熵损失

- 训练轮数：10
- 批次大小：64
- 随机种子：42
- 数据加载工作进程数：2
- 日志记录间隔：每 100 个批次

上述 8 种超参数组合分别在两种模型上训练，共计 16 组实验。

3 实验结果与分析

3.1 总体结果

表 1 展示了所有 16 组实验在 MNIST 测试集上的最佳准确率。每列最优结果已加粗标出。

表 1: 不同超参数配置与模型组合下的测试集最佳准确率

编号	学习率	权重衰减	丢弃率	基础 CNN	深层 CNN
1	0.0005	0.0	0.3	0.9927	0.9957
2	0.0005	0.0001	0.3	0.9933	0.9948
3	0.001	0.0	0.3	0.9932	0.9951
4	0.001	0.0001	0.3	0.9925	0.9952
5	0.0005	0.0	0.5	0.9937	0.9957
6	0.0005	0.0001	0.5	0.9926	0.9951
7	0.001	0.0	0.5	0.9932	0.9947
8	0.001	0.0001	0.5	0.9925	0.9940
基础 CNN 最佳				0.9937	—
深层 CNN 最佳				—	0.9957

3.2 结果分析

3.2.1 模型架构对性能的影响

从表 1 可以清晰地看出，**深层网络在所有 8 组超参数配置下均优于基础网络**。深层网络的最佳准确率为 0.9957，比基础网络的最佳准确率 0.9937 高出 0.20 个百分点。考虑到 MNIST 数据集的准确率已经处于较高水平 ($> 99\%$)，这一提升是显著的。

深层网络的性能优势主要来源于：

1. **更深的网络结构**：5 个卷积层（相比之下基础网络仅 2 个）提供了更强的特征提取能力，能够学习更抽象、更具判别力的特征；
2. **批量归一化**：使各层输入的分布更加稳定，允许使用更大的学习率并加速收敛，同时具有一定的正则化效果；
3. **更宽的特征图**：通道数从基础网络的 64 扩展到 128，使模型能表示更丰富的视觉模式。

3.2.2 超参数对性能的影响

对基础网络而言：

- 较低的学习率（0.0005）整体表现优于较高的学习率（0.001），说明浅层网络对学习率更为敏感；
- 不使用权重衰减的配置普遍优于使用权重衰减（0.0001）的对应配置；
- 在学习率为 0.0005、不使用权重衰减的条件下，较高的丢弃率（0.5）取得了该模型的最佳结果（0.9937），而较低的丢弃率（0.3）则为 0.9927，差异不大。

对深层网络而言：

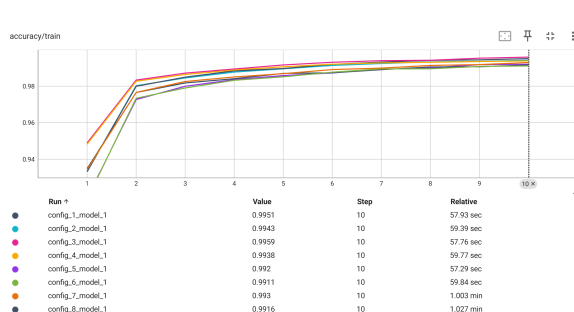
- 模型对超参数变化表现出更强的鲁棒性：8 组配置的准确率波动范围为 0.9940–0.9957（极差 0.0017），而基础网络的波动范围为 0.9925–0.9937（极差 0.0012）；
- 不使用权重衰减时准确率普遍更高，说明批量归一化本身已提供足够的正则化效果，额外的权重衰减反而可能限制模型容量；
- 最优配置出现在学习率为 0.0005 且不使用权重衰减的条件下（表 1 中编号 1 与 5），两者均达到 0.9957，表明在该条件下深层网络对丢弃率（0.3 与 0.5）不敏感。

3.3 训练过程准确率曲线

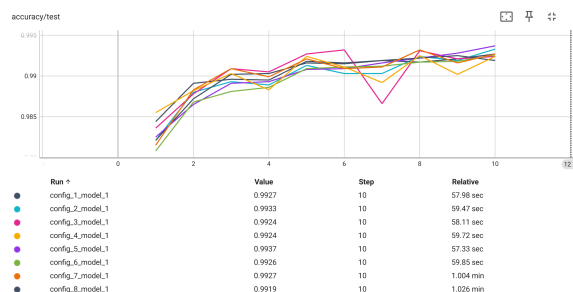
图 1 和图 2 分别展示了基础 CNN 和深层 CNN 在 8 组超参数配置下的训练与测试准确率随训练轮数的变化趋势。图中曲线直接截取自 TensorBoard 可视化面板，图例中的标签对应于不同超参数配置的实验运行名称，其与表 1 中编号的对应关系及具体超参数取值见表 2。

表 2: TensorBoard 图例标签与超参数配置的对应关系

图例标签 (基础 CNN)	图例标签 (深层 CNN)	表 1 编号	超参数
config_1_model_1	config_1_model_2	1	lr=0.0005, wd=0.0, dropout=0.3
config_2_model_1	config_2_model_2	2	lr=0.0005, wd=0.0001, dropout=0.3
config_3_model_1	config_3_model_2	3	lr=0.001, wd=0.0, dropout=0.3
config_4_model_1	config_4_model_2	4	lr=0.001, wd=0.0001, dropout=0.3
config_5_model_1	config_5_model_2	5	lr=0.0005, wd=0.0, dropout=0.5
config_6_model_1	config_6_model_2	6	lr=0.0005, wd=0.0001, dropout=0.5
config_7_model_1	config_7_model_2	7	lr=0.001, wd=0.0, dropout=0.5
config_8_model_1	config_8_model_2	8	lr=0.001, wd=0.0001, dropout=0.5



(a) 训练准确率



(b) 测试准确率

图 1: 基础 CNN (双卷积层结构) 在 8 组超参数配置下的准确率曲线。图例标签与超参数配置的对应关系见表 2。

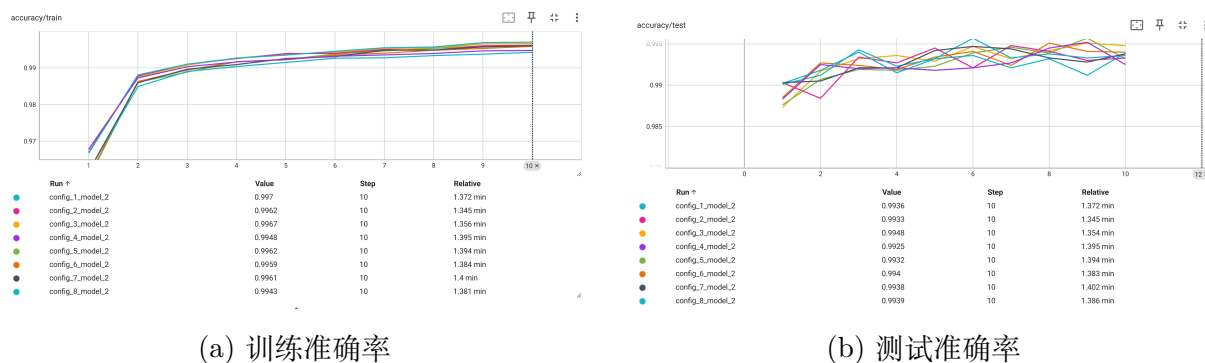


图 2: 深层 CNN (五卷积层 + 批量归一化结构) 在 8 组超参数配置下的准确率曲线。图例标签与超参数配置的对对应关系见表 2。

从图中可以观察到:

1. 深层网络的训练准确率收敛速度明显快于基础网络: 在首个训练轮次结束时, 深层网络的训练准确率已达到约 97% 以上, 而基础网络仅达到约 92%–94%;
2. 深层网络的测试准确率曲线更为平稳, 不同超参数配置间的差异较小, 再次印证了其对超参数变化具有较强的鲁棒性;
3. 基础网络在训练准确率上存在较明显的配置间差异, 尤其是使用较高学习率(0.001)时, 收敛速度明显慢于低学习率(0.0005)的配置;
4. 两种模型的测试准确率均未出现明显的过拟合迹象(训练准确率与测试准确率未出现显著背离), 说明模型容量与数据集规模匹配良好。

4 调试与错误排查

本实验在代码中实现了完善的日志记录机制, 便于调试与错误排查。调试信息通过两种方式记录:

1. **文本日志:** 使用 Python 标准库 logging 模块, 为每个实验运行创建独立的日志文件。日志包含实验启动信息(模型名称、配置文件路径、日志路径)、每个训练轮次的训练损失、训练准确率、测试损失和测试准确率、以及最终的最佳准确率和模型保存路径。所有日志文件存储在 logs/ 目录下, 文件命名格式为 config_{i}_model_{j}.log (其中 $i \in \{1, \dots, 8\}$ 对应超参数组合编号, $j = 1$ 对应基础 CNN, $j = 2$ 对应深层 CNN)。
2. **TensorBoard 可视化:** 使用 PyTorch 的 SummaryWriter 将训练过程中的标量指标(批次级训练损失、训练轮次级训练损失、训练准确率、测试损失、测试准确率)实时写入 TensorBoard 事件文件, 存储在 runs/ 目录下。可通过 tensorboard --logdir=runs 命令启动可视化面板, 直观对比不同实验的训练曲线。

在实际运行过程中，如果实验出现错误（例如中途崩溃、GPU 内存不足、数据加载异常等），可以通过以下方式进行排查：

- 查看对应日志文件中的错误信息和最后一条记录，确定崩溃发生的训练轮次和批次位置；
- 检查 TensorBoard 中的损失曲线是否出现异常（如突然飙升或变为 NaN），判断是否存在梯度爆炸等问题；
- 验证模型检查点（`model/*.pth`）是否完整保存，若训练提前中断，可从最近的检查点恢复。

这一日志系统确保了实验的可追溯性和可复现性，是深度学习工程实践中的重要组成部分。

5 自定义手写数字识别实验

在完成 MNIST 测试集上的模型对比后，本文进一步使用自行书写并拍摄的数字图像对训练好的模型进行推理测试。该实验用于检验模型在非标准 MNIST 图像上的泛化表现，同时验证训练模型在实际输入场景中的可用性。自定义测试图像存放于 `numbersbyHAND/` 目录下，文件名为 `0.png` 至 `9.png`，分别对应人工书写的数字 0–9。

5.1 模型选择与预测方法

根据训练日志 `logs/config_*.log` 中记录的最佳测试准确率，深层 CNN 在 `config_1_model_2` 和 `config_5_model_2` 两组配置下均取得了最高测试准确率 99.57%。本文选取 `config_5_model_2.yaml` 对应的检查点 `model/mnist_cnn_config_5_model_2.pth` 作为自定义图像识别模型。预测脚本首先根据配置文件构建相同结构的 MNISTCNN2 网络，随后加载保存的 `model_state`，并对每张输入图像输出预测类别及 softmax 置信度。

由于自定义图像来自手机或截图拍摄，其分布与 MNIST 数据集存在明显差异：MNIST 图像为黑色背景上的白色数字，而自定义图像通常包含灰色纸面背景、光照变化和较大的空白边缘。若仅将原图直接缩放至 28×28 ，背景信息会被一并压缩进输入张量，导致模型容易将多个数字误判为形状更复杂的类别（如数字 8）。因此，预测阶段采用了与 MNIST 形式更一致的预处理流程：

1. 将输入图像转换为灰度图，并通过自动对比度增强扩大前景与背景的灰度差异；
2. 对深色笔迹图像进行反色，使数字笔迹转化为亮色前景；
3. 使用阈值二值化去除灰色背景，仅保留主要笔迹区域；
4. 根据非零像素的包围盒裁剪数字主体，并按比例缩放至 20×20 ；

5. 将缩放后的数字居中粘贴到 28×28 的黑色画布上；
6. 使用 MNIST 数据集的均值 0.1307 和标准差 0.3081 进行归一化。

这一处理流程保持了 MNIST 数据集中“居中、黑底、亮色数字”的输入形式，从而降低了训练分布与实际预测图像分布之间的差异。此外，预测脚本中的模型检查点参数设置为可选参数；当命令行未显式指定检查点时，脚本会读取配置文件中 `predict.checkpoint` 指定的最优模型路径，避免配置文件与实际加载模型不一致的问题。

5.2 识别结果

使用修正后的预处理流程，对 10 张自定义手写数字图像进行预测，运行命令如下：

```
python predict_handwritten.py --config config/config_5_model_2.yaml
```

识别结果如表 3 所示。

表 3: 自定义手写数字识别结果

图像文件	真实类别	预测类别	置信度
0.png	0	0	1.0000
1.png	1	1	1.0000
2.png	2	2	0.9999
3.png	3	3	1.0000
4.png	4	4	0.9997
5.png	5	5	0.9999
6.png	6	6	0.9999
7.png	7	7	0.8076
8.png	8	8	1.0000
9.png	9	9	0.8766

结果表明，经过针对实际图像分布的预处理后，模型能够正确识别全部 10 张自定义手写数字图像。其中数字 7 和数字 9 的置信度相对较低，说明这两类样本的书写形态与训练集中的典型样本仍存在一定差异；但总体而言，深层 CNN 在经过适当输入规范化后可以较好地迁移到简单的真实手写图像识别场景。该实验也说明，对于图像分类模型而言，推理阶段的输入预处理与训练数据分布的一致性是影响实际识别效果的重要因素。

6 结论

本实验通过在 MNIST 手写数字识别任务上对两种卷积神经网络架构和八种超参数配置进行系统对比，得出以下结论：

1. **深层网络显著提升性能**：引入批量归一化的五卷积层深层网络在所有超参数配置下均优于双卷积层基础网络，最佳准确率达到 99.57%，相比基础网络的 99.37% 有显著提升；
2. **低学习率表现更优**：当学习率为 0.0005 时，两种模型的整体表现均优于学习率为 0.001 时；
3. **批量归一化提供隐含正则化**：深层网络在不使用权重衰减时表现最佳，说明批量归一化本身已具备一定的正则化效果，额外的权重衰减可能限制模型容量；
4. **网格搜索是有效的超参数调优方法**：通过 $2 \times 2 \times 2$ 的网格搜索，成功定位了各模型的最优超参数组合。

未来的工作可以进一步探索数据增强策略、学习率调度器以及更现代的网络结构（如残差连接）对性能的影响。

A 附录：完整代码

本项目的完整代码结构如下：

CNN-MNIST/

model.py	# 模型定义（基础 CNN 与深层 CNN）
dataloader.py	# 数据加载与预处理
train.py	# 训练与评估循环
main.py	# 主入口，解析配置、训练、保存
log.py	# 日志配置
predict_handwritten.py	# 自定义手写数字预测
run_experiments.sh	# 批量实验脚本
config/	# 16 个 YAML 配置文件
logs/	# 训练日志输出
model/	# 保存的模型检查点
runs/	# TensorBoard 事件文件
numbersbyHAND/	# 手写数字测试图像
src/	# TensorBoard 截图等素材
report.tex	# 本报告 LaTeX 源文件

A.1 model.py ——模型定义

```
1 import torch
2 from torch import nn
3
4
5 class MNISTCNN1(nn.Module):
6     """Two-layer CNN with a single fully-connected classifier."""
7     def __init__(
8         self,
9         conv1_channels: int = 32,
10        conv2_channels: int = 64,
11        hidden_size: int = 128,
12        dropout: float = 0.5,
13    ) -> None:
14        super().__init__()
15
16        self.features = nn.Sequential(
17            nn.Conv2d(1, conv1_channels, kernel_size=3, padding=1),
18            nn.ReLU(),
19            nn.MaxPool2d(2),
20            nn.Conv2d(conv1_channels, conv2_channels, kernel_size=3,
padding=1),
21            nn.ReLU(),
22            nn.MaxPool2d(2),
23        )
24
25        self.classifier = nn.Sequential(
26            nn.Flatten(),
27            nn.Linear(conv2_channels * 7 * 7, hidden_size),
28            nn.ReLU(),
29            nn.Dropout(dropout),
30            nn.Linear(hidden_size, 10),
31        )
32
33    def forward(self, x: torch.Tensor) -> torch.Tensor:
34        x = self.features(x)
35        return self.classifier(x)
36
37
```

```
38 class MNISTCNN2(nn.Module):
39     """Deeper CNN with batch normalization and three conv blocks."""
40
41     def __init__(
42         self,
43         conv1_channels: int = 32,
44         conv2_channels: int = 64,
45         conv3_channels: int = 128,
46         hidden_size: int = 256,
47         dropout: float = 0.5,
48     ) -> None:
49         super().__init__()
50
51         self.block1 = nn.Sequential(
52             nn.Conv2d(1, conv1_channels, kernel_size=3, padding=1),
53             nn.BatchNorm2d(conv1_channels),
54             nn.ReLU(),
55             nn.Conv2d(conv1_channels, conv1_channels, kernel_size=3,
padding=1),
56             nn.BatchNorm2d(conv1_channels),
57             nn.ReLU(),
58             nn.MaxPool2d(2),
59         )
60
61         self.block2 = nn.Sequential(
62             nn.Conv2d(conv1_channels, conv2_channels, kernel_size=3,
padding=1),
63             nn.BatchNorm2d(conv2_channels),
64             nn.ReLU(),
65             nn.Conv2d(conv2_channels, conv2_channels, kernel_size=3,
padding=1),
66             nn.BatchNorm2d(conv2_channels),
67             nn.ReLU(),
68             nn.MaxPool2d(2),
69         )
70
71         self.block3 = nn.Sequential(
72             nn.Conv2d(conv2_channels, conv3_channels, kernel_size=3,
padding=1),
73             nn.BatchNorm2d(conv3_channels),
```

```

74         nn.ReLU(),
75         nn.MaxPool2d(2),
76     )
77
78     self.classifier = nn.Sequential(
79         nn.Flatten(),
80         nn.Linear(conv3_channels * 3 * 3, hidden_size),
81         nn.ReLU(),
82         nn.Dropout(dropout),
83         nn.Linear(hidden_size, 10),
84     )
85
86     def forward(self, x: torch.Tensor) -> torch.Tensor:
87         x = self.block1(x)
88         x = self.block2(x)
89         x = self.block3(x)
90         return self.classifier(x)
91
92
93 # Map config model names to their class definitions
94 MODEL_REGISTRY: dict[str, type[nn.Module]] = {
95     "MNISTCNN1": MNISTCNN1,
96     "MNISTCNN2": MNISTCNN2,
97 }
98
99
100 def get_model_by_name(name: str) -> type[nn.Module]:
101     """Look up a model class by its registry name."""
102     if name not in MODEL_REGISTRY:
103         raise KeyError(
104             f"Unknown model '{name}'. "
105             f"Available: {list(MODEL_REGISTRY.keys())}"
106         )
107     return MODEL_REGISTRY[name]

```

Listing 1: model.py

A.2 dataloader.py —— 数据加载

```

1 from typing import Any

```

```
2 from torch.utils.data import DataLoader
3 from torchvision import datasets, transforms
4
5
6 def build_dataloaders(
7     config: dict[str, Any]
8 ) -> tuple[DataLoader, DataLoader]:
9     """Create train and test DataLoaders from a YAML config."""
10    data_config = config["data"]
11    data_root = data_config["root"]
12
13    transform = transforms.Compose(
14        [
15            transforms.ToTensor(),
16            transforms.Normalize((0.1307,), (0.3081,)), # MNIST mean
17            and std
18        ]
19    )
20
21    train_dataset = datasets.MNIST(
22        root=data_root,
23        train=True,
24        download=True,
25        transform=transform,
26    )
27
28    test_dataset = datasets.MNIST(
29        root=data_root,
30        train=False,
31        download=True,
32        transform=transform,
33    )
34
35    train_loader = DataLoader(
36        train_dataset,
37        batch_size=data_config["batch_size"],
38        shuffle=True,
39        num_workers=data_config["num_workers"],
40    )
```

```
41     test_loader = DataLoader(  
42         test_dataset,  
43         batch_size=data_config["batch_size"],  
44         shuffle=False,  
45         num_workers=data_config["num_workers"],  
46     )  
47  
48     return train_loader, test_loader
```

Listing 2: dataloader.py

A.3 train.py ——训练与评估

```
1 import torch  
2 from torch import nn  
3 from torch.utils.data import DataLoader  
4 from torch.utils.tensorboard import SummaryWriter  
5  
6  
7 def train_one_epoch(  
8     model: nn.Module,  
9     loader: DataLoader,  
10    loss_fn: nn.Module,  
11    optimizer: torch.optim.Optimizer,  
12    device: torch.device,  
13    writer: SummaryWriter,  
14    epoch: int,  
15    log_interval: int,  
16 ) -> tuple[float, float]:  
17     """Run one training epoch and return average loss and accuracy.  
18     """  
19     model.train()  
20     total_loss = 0.0  
21     correct = 0  
22  
23     for batch_idx, (images, labels) in enumerate(loader):  
24         images = images.to(device)  
25         labels = labels.to(device)  
26  
27         optimizer.zero_grad()
```



```
27     logits = model(images)
28     loss = loss_fn(logits, labels)
29     loss.backward()
30     optimizer.step()
31
32     total_loss += loss.item()
33     predictions = logits.argmax(dim=1)
34     correct += (predictions == labels).sum().item()
35
36     # Log per-batch loss at a fixed interval for TensorBoard
37     step = (epoch - 1) * len(loader) + batch_idx
38     if batch_idx % log_interval == 0:
39         writer.add_scalar("loss/train_batch", loss.item(), step)
40
41     avg_loss = total_loss / len(loader)
42     accuracy = correct / len(loader.dataset) # type: ignore
43     writer.add_scalar("loss/train_epoch", avg_loss, epoch)
44     writer.add_scalar("accuracy/train", accuracy, epoch)
45     return avg_loss, accuracy
46
47
48 def evaluate(
49     model: nn.Module,
50     loader: DataLoader,
51     loss_fn: nn.Module,
52     device: torch.device,
53 ) -> tuple[float, float]:
54     """Evaluate the model on a dataset without gradient updates."""
55     model.eval()
56     total_loss = 0.0
57     correct = 0
58
59     with torch.no_grad():
60         for images, labels in loader:
61             images = images.to(device)
62             labels = labels.to(device)
63
64             logits = model(images)
65             loss = loss_fn(logits, labels)
66             total_loss += loss.item()
```

```
67
68     predictions = logits.argmax(dim=1)
69     correct += (predictions == labels).sum().item()
70
71     avg_loss = total_loss / len(loader)
72     accuracy = correct / len(loader.dataset) # type: ignore
73     return avg_loss, accuracy
```

Listing 3: train.py

A.4 main.py ——主程序入口

```
1 import argparse
2 from pathlib import Path
3
4 import torch
5 from torch import nn
6 from torch.utils.tensorboard import SummaryWriter
7 import yaml
8
9 from log import setup_logging, get_model_logger
10 from dataloader import build_dataloaders
11 from train import train_one_epoch, evaluate
12 from model import get_model_by_name
13
14
15 def main() -> None:
16     parser = argparse.ArgumentParser()
17     parser.add_argument("--config", default="config/config_1_model_1.
18         yaml")
19
20     args = parser.parse_args()
21
22     config_path = args.config
23
24     with open(config_path, "r", encoding="utf-8") as f:
25         config = yaml.safe_load(f)
26
27     # Separate model name from constructor kwargs
28     model_config = dict(config["model"])
29     model_name = model_config.pop("model_name", "MNISTCNN1")
```

```
28
29     logger = get_model_logger(model_name)
30     log_path = Path(config["train"]["log_path"])
31     setup_logging(log_path, logger)
32
33     logger.info(f"Experiment started")
34     logger.info(f"Model: {model_name}")
35     logger.info(f"Config: {config_path}")
36     logger.info(f"Log file: {log_path}")
37     logger.info("-" * 60)
38
39     torch.manual_seed(config["seed"])
40     if torch.cuda.is_available():
41         torch.cuda.manual_seed_all(config["seed"])
42     device = torch.device("cuda" if torch.cuda.is_available() else "
cpu")
43
44     train_loader, test_loader = build_dataloaders(config)
45
46     model_cls = get_model_by_name(model_name)
47     model = model_cls(**model_config).to(device)
48
49     loss_fn = nn.CrossEntropyLoss()
50     optimizer = torch.optim.Adam(
51         model.parameters(),
52         lr=config["train"]["learning_rate"],
53         weight_decay=config["train"]["weight_decay"],
54     )
55
56     log_dir = config["train"]["tensorboard_dir"]
57
58     save_dir = Path(config["train"]["save_dir"])
59     save_dir.mkdir(parents=True, exist_ok=True)
60
61     checkpoint_path = save_dir / config["train"]["save_name"]
62     best_accuracy = 0.0
63
64     with SummaryWriter(log_dir=log_dir) as writer:
65         for epoch in range(1, config["train"]["epochs"] + 1):
66             train_loss, train_accuracy = train_one_epoch(
```

```

67         model=model,
68         loader=train_loader,
69         loss_fn=loss_fn,
70         optimizer=optimizer,
71         device=device,
72         writer=writer,
73         epoch=epoch,
74         log_interval=config["train"]["log_interval"],
75     )
76     test_loss, test_accuracy = evaluate(model, test_loader,
loss_fn, device)
77
78     writer.add_scalar("loss/test", test_loss, epoch)
79     writer.add_scalar("accuracy/test", test_accuracy, epoch)
80
81     logger.info(
82         f"Epoch {epoch:02d}/{config['train']['epochs']} "
83         f"train_loss={train_loss:.4f} "
84         f"train_acc={train_accuracy:.4f} "
85         f"test_loss={test_loss:.4f} "
86         f"test_acc={test_accuracy:.4f}"
87     )
88
89     if test_accuracy > best_accuracy:
90         best_accuracy = test_accuracy
91         torch.save(
92             {
93                 "model_state": model.state_dict(),
94                 "config": config,
95                 "accuracy": best_accuracy,
96             },
97             checkpoint_path,
98         )
99
100     logger.info(f"Best test accuracy: {best_accuracy:.4f}")
101     logger.info(f"Model saved to: {checkpoint_path}")
102     logger.info(f"TensorBoard logs: {log_dir}")
103
104
105 if __name__ == "__main__":

```

```
main()
```

Listing 4: main.py

A.5 log.py ——日志模块

```
1 import logging
2 from logging import Logger
3 from pathlib import Path
4
5
6 def setup_logging(log_path: Path, logger: Logger) -> None:
7     """Configure a logger to write INFO messages to a file."""
8     log_path.parent.mkdir(parents=True, exist_ok=True)
9
10    logger.setLevel(logging.INFO)
11
12    file_handler = logging.FileHandler(str(log_path), mode="w",
13    encoding="utf-8")
14    file_handler.setLevel(logging.INFO)
15
16    formatter = logging.Formatter(
17        fmt="%(asctime)s %(message)s",
18        datefmt="%Y-%m-%d %H:%M:%S",
19    )
20    file_handler.setFormatter(formatter)
21
22    logger.addHandler(file_handler)
23
24    logger.propagate = False
25
26 def get_model_logger(model_name: str) -> logging.Logger:
27     """Return a named logger for a specific model."""
28     return logging.getLogger(f"MNIST_CNN_{model_name}")
```

Listing 5: log.py

A.6 predict_handwritten.py ——手写数字预测

```
1 import argparse
2 from pathlib import Path
3
4 import torch
5 from PIL import Image, ImageOps
6 from torchvision import transforms
7 import yaml
8
9 from model import get_model_by_name
10
11
12 def preprocess_image(path: Path, invert: bool) -> torch.Tensor:
13     image = Image.open(path).convert("L")
14     image = ImageOps.autocontrast(image)
15     if invert:
16         image = ImageOps.invert(image)
17     image = image.point(lambda pixel: 255 if pixel > 100 else 0)
18
19     bbox = image.getbbox()
20     if bbox is not None:
21         image = image.crop(bbox)
22     image.thumbnail((20, 20), Image.Resampling.LANCZOS)
23     canvas = Image.new("L", (28, 28), 0)
24     canvas.paste(image, ((28 - image.width) // 2, (28 - image.height)
25 // 2))
26
27     transform = transforms.Compose(
28         [
29             transforms.ToTensor(),
30             transforms.Normalize((0.1307,), (0.3081,)),
31         ]
32     )
33
34     return transform(canvas).unsqueeze(0) # type: ignore
35
36 def main() -> None:
37     parser = argparse.ArgumentParser(description="Predict handwritten
38 digit images.")
```

```
38     parser.add_argument("--config", default="config/config_1_model_1.
39     yaml")
40     parser.add_argument("--image-dir", default="numbersbyHAND")
41     parser.add_argument("--checkpoint", default=None)
42     args = parser.parse_args()
43
44     base_dir = Path(__file__).resolve().parent
45     config_path = Path(args.config)
46     if not config_path.is_absolute():
47         config_path = base_dir / config_path
48     with config_path.open("r", encoding="utf-8") as f:
49         config = yaml.safe_load(f)
50         predict_config = config["predict"]
51
52     image_dir = Path(args.image_dir or predict_config["image_dir"])
53     if not image_dir.is_absolute():
54         image_dir = base_dir / image_dir
55
56     checkpoint_path = Path(args.checkpoint or predict_config["
57     checkpoint"])
58     if not checkpoint_path.is_absolute():
59         checkpoint_path = base_dir / checkpoint_path
60     image_paths = sorted(
61         [
62             *image_dir.glob("*.png"),
63             *image_dir.glob("*.jpg"),
64             *image_dir.glob("*.jpeg"),
65         ]
66     )
67
68     if not checkpoint_path.exists():
69         raise FileNotFoundError(f"Model checkpoint not found: {
70     checkpoint_path}")
71     if not image_paths:
72         raise FileNotFoundError(f"No png/jpg images found in: {
73     image_dir}")
74
75     device = torch.device("cuda" if torch.cuda.is_available() else "
76     cpu")
```

```

73     model_config = dict(config["model"])
74     model_name = model_config.pop("model_name", "MNISTCNN1")
75     model = get_model_by_name(model_name)(**model_config).to(device)
76     checkpoint = torch.load(checkpoint_path, map_location=device)
77     model.load_state_dict(checkpoint["model_state"])
78     model.eval()
79
80     for image_path in image_paths:
81         image = preprocess_image(image_path, invert=predict_config["
invert"]).to(device)
82         with torch.no_grad():
83             probabilities = torch.softmax(model(image), dim=1)
84             confidence, prediction = probabilities.max(dim=1)
85
86             print(f"{image_path.name}: {prediction.item()} confidence={
confidence.item():.4f}")
87
88
89 if __name__ == "__main__":
90     main()

```

Listing 6: predict_handwritten.py

A.7 run_experiments.sh —— 批量实验脚本

```

1  #!/bin/sh
2  set -e
3
4  cd "$(dirname "$0")"
5
6  for model in 1 2; do
7      i=1
8      while [ "$i" -le 8 ]; do
9          config_file="config/config_${i}_model_${model}.yaml"
10         echo "Running experiment config_${i}_model_${model}: ${
config_file}"
11         python main.py --config "$config_file"
12         i=$((i + 1))
13     done
14 done

```



```
15  
16 echo "All experiments finished."
```

Listing 7: run_experiments.sh